

Project 3

Prathik Kumar Gaddam (G27195400), Jatin Chopra (G41931788), Vinod Kumar (G39671299), and
Rohith Yellambalase Vijayakumar (G26286080)

1 Problem Statement

The diameter of a graph is defined as the largest distance between any pair of vertices of G . Give an efficient polynomial algorithm to find the diameter of the graph.

2 Problem Analysis

Objective: The diameter of a graph is the maximum shortest path distance between any pair of vertices. We aim to design an efficient algorithm to compute this diameter using dynamic programming. We use the Floyd-Warshall algorithm, a dynamic programming method to find shortest paths between all pairs of vertices. The algorithm iteratively checks if any vertex can act as an intermediate to shorten paths between two other vertices.

Assumptions: The graph is connected, with Non-negative edge weights (no negative cycles), an Undirected graph (edges have equal weight in both directions), and No self-loops with weight > 0 ; diagonal entries are 0.

3 Implementation Characteristics

Steps to implement the diameter of the graph using dynamic programming

1. Graph Representation

We represent the graph using an adjacency matrix. To initialize the distance matrix $dist$ with dimensions $n \times n$ (where n is the number of vertices), we set all values to $float('inf')$ to indicate no direct paths. For each **vertex** u , we set the distance to itself as $dist[u][u] = 0$.

2. Input Edge Weight

For each edge (u, v) with weight w , we update the distance matrix by iterating over each node. For each node u , we look at its edges and set the corresponding weights in the distance matrix, using $dist[u][v] = weight$.

3. Apply Floyd-Warshall Algorithm

We use three nested loops to update the distance matrix. For each intermediate vertex k , we iterate through each start vertex i and end vertex j , checking if the distance $dist[i][j]$ can be minimized by going through the intermediate vertex. We do this with the expression $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$.

4. Calculate the Diameter

After executing the **Floyd-Warshall algorithm**, we calculate the diameter by initializing a variable **diameter** to 0. We iterate through the distance matrix, and for each pair of **vertices** (i, j) , if $dist[i][j]$ is less than $float('inf')$, we update the diameter using $diameter = \max(diameter, dist[i][j])$.

Theoretical time analysis

The Floyd-Warshall algorithm is characterized by its time complexity, which can be analyzed as follows:

1. Initialization:

We initialize a distance matrix $dist$ of size $n \times n$. This operation takes $O(n^2)$ time, as we need to set the initial values for all pairs of vertices.

2. Main Algorithm:

The core of the Floyd-Warshall algorithm has three nested loops: the outer loop for each intermediate vertex k , the middle for each start vertex i , and the innermost for each end vertex j , all iterating $O(n)$ times. Thus, the total complexity is $O(n^3)$.

3. Diameter Calculation:

After calculating the shortest paths, we compute the diameter by iterating through the distance matrix to find the maximum distance. This operation also takes $O(n^2)$ time.

4. Overall Time Complexity:

Combining the complexities of all parts:

- Initialization: $O(n^2)$, Main Algorithm: $O(n^3)$, Diameter Calculation: $O(n^2)$

- Thus, the overall time complexity of the Floyd-Warshall algorithm for computing the diameter of a graph is dominated by the main algorithm, leading to **Total Time Complexity= $O(n^3)$** .

4 Experimental Analysis

4.1 Program Listing

- We have computed the execution time using n values = [100, 200, 300, 400, 500].

- **Pseudocode for Diameter of graph**

```
function floydWarshall(graph)
    n = number of vertices in graph
    dist = array of size n x n initialized to float('inf')
    for u from 0 to n-1:
        dist[u][u] = 0
    for each edge (u, v) in graph:
        dist[u][v] = weight of edge (u, v)
    for k from 0 to n-1:
        for i from 0 to n-1:
            for j from 0 to n-1:
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    diameter = 0
    for i from 0 to n-1:
        for j from 0 to n-1:
            if dist[i][j] < float('inf'):
                diameter = max(diameter, dist[i][j]) // Find
    return diameter // Return the diameter of the graph
```

Github link for the project code:

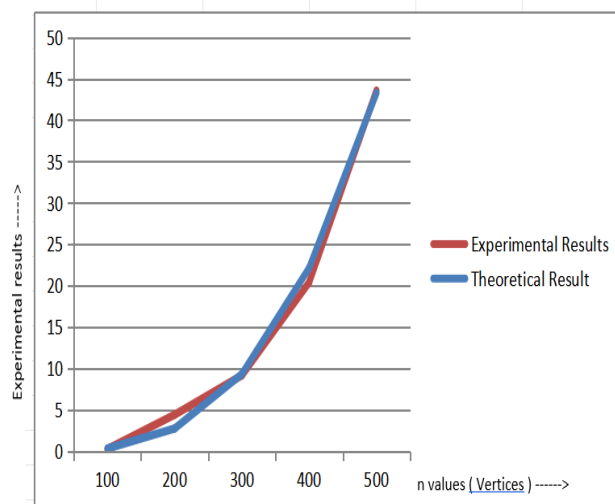
4.2 Data Normalization Notes

Data Normalization, we must find the Scaling constant to convert the theoretical log values into time in seconds. For n values 100, 200, 300, 400, and 500, since time complexity is $O(n^3)$ therefore theoretical times will be thrice of the n values and Experimental Result(seconds) – 0.292201, 4.432437, 9.193199, 20.424332, 43.564433. **Scaling constant = average experimental result / average theoretical = 15.5813204 / 45000000 = 3.4625156×10^{-7} .**

4.3 Output Numerical Data

n values (no. of vertices)	Experimental Result(seconds)	Theoretical Result $O(n^3)$	Adjusted theoretical result value = Scaling constant * Theoretical result
100	0.292201	1000000	$3.4625156 \times 10^{-7} \times 100 = 0.34625156$
200	4.432437	8000000	$3.4625156 \times 10^{-7} \times 200 = 2.770012516$
300	9.193199	27000000	$3.4625156 \times 10^{-7} \times 300 = 9.34879224$
400	20.424332	64000000	$3.4625156 \times 10^{-7} \times 400 = 22.16010012$
500	43.564433	125000000	$3.4625156 \times 10^{-7} \times 500 = 43.28144556$

4.4 Graph



4.5 Graph Observations

Asymptotic Complexity Analysis:

The Floyd-Warshall algorithm has a time complexity of $O(n^3)$, with runtime growing cubically as the vertex count n increases. This reflects the algorithm's three nested loops, each iterating n times. While efficient for smaller graphs, this cubic growth makes it computationally intensive for large graphs, underscoring the trade-off between accuracy and computational cost.

Alignment of Theoretical and Experimental Curves:

Experimental results align closely with the $O(n^3)$ theoretical curve, indicating optimal implementation and consistent performance. The minimal variance between the two suggests predictability, making the algorithm suitable for accurate shortest-path calculations, provided the graph size is manageable.

5 Conclusions

The Floyd-Warshall algorithm demonstrates strong performance in computing the graph diameter, with its theoretical time complexity of $O(n^3)$ aligning well with both experimental and theoretical results. This close correspondence indicates that the implementation achieves optimal efficiency, making it a reliable choice for small to moderately-sized networks. However, due to its cubic complexity, the algorithm becomes increasingly computationally intensive for larger graphs. Consequently, for extensive networks, it may be advisable to explore alternative algorithms or optimizations to effectively manage the exponential increase in computation time while still ensuring accuracy.